

Relazione Progetto Laboratorio II: MapReduce in C POSIX

1. Introduzione e Obiettivi

Questo progetto ha lo scopo di realizzare in linguaggio C un framework concorrente e multiprocesso che implementi il modello MapReduce. Tutto il codice è stato sviluppato attenendosi allo standard C11 (<threads.h>) per la gestione del multithreading e allo standard POSIX per l'infrastruttura multiprocesso e la relativa comunicazione.

Gli obiettivi principali del progetto sono stati:

- **Esecuzione in parallelo:** Il carico di lavoro è stato suddiviso istanziando processi separati (tramite `fork()`) per coordinare, mappare e ridurre i dati.
- **Comunicazione sicura tra processi:** I processi comunicano tra loro scambiandosi i dati attraverso le pipe. Al fine di assicurare la consistenza dei dati, è stato implementato un protocollo di comunicazione che struttura i messaggi nel formato `{lunghezza_token},{token}` e `{lunghezza_valore},{valore}`.
- **Gestione opaca dei dati:** Il framework è progettato per operare in modo indipendente rispetto alla natura del risultato stabilita dall'utente. Poiché il framework non possiede conoscenze semantiche sul dato emesso, esso viene manipolato strettamente come dato opaco. Il risultato finale viene serializzato su file traducendo i byte raw in formato esadecimale.
- **Resistenza agli errori:** Alla ricezione del segnale `SIGINT`, il processo principale gestisce la terminazione pulita di tutti i processi figli e la corretta chiusura dei descrittori. L'assenza di memory leak è stata verificata tramite i Sanitizers.

2. Interfaccia Pubblica

Il framework è esposto all'utente tramite una libreria statica (`libmr.a`), la cui interfaccia pubblica richiede un'interazione minimale. L'utente interagisce con il sistema popolando una struttura dati di configurazione opaca (`mr_t`), all'interno della quale inietta i puntatori a funzione per le proprie logiche:

- **mr_mapper_t**: La callback invocata dai worker del Mapper per processare una specifica riga testuale in ingresso.
- **mr_reducer_t**: La callback invocata dai worker del Reducer per iterare e collassare l'array di risultati accomunati dal medesimo token.

Per l'emissione e il consolidamento dei dati, il framework espone le funzioni `emit_pair` e `emit_result`. Una volta preparata l'istanza **mr_t**, il ciclo di vita dell'intero flusso viene innescato tramite l'entry-point pubblico `mr_start()`.

3. Architettura del Sistema

L'architettura si basa su un pattern "Master-Worker" suddiviso in tre processi principali: Main Process, Processo Mapper e Processo Reducer.

Quando l'esecuzione viene avviata, il Main Process predispone le pipe di comunicazione e istanzia i processi Mapper e Reducer tramite la system call `fork()`. Per isolare i flussi comunicativi, i processi figli impiegano la chiamata `dup2()` per sovrascrivere `STDIN` e `STDOUT`.

Mentre Mapper e Reducer operano in background, il Main process acquisisce le directory, esegue il parsing dei file di input e invia le singole righe al Mapper. Al termine, il Main utilizza `waitpid()` per attendere la terminazione dei figli, raccoglie i risultati aggregati, li persiste sul file di output ed esegue il cleanup.

La Comunicazione (Pipe)

Per la comunicazione inter-processo vengono impiegate tre pipe unidirezionali:

- **main_to_mapper**: Il Main invia le singole righe al Mapper.
- **mapper_to_reducer**: Il Mapper invia le coppie processate (`[token, valore]`) al Reducer.
- **reducer_to_main**: Il Reducer invia i risultati finali aggregati al Main.

4. Modulo Mapper

Il modulo Mapper è responsabile della prima fase di elaborazione. La sua architettura interna sfrutta un pool di thread gestiti tramite lo standard C11 per processare concorrentemente le righe di input.

Gestione dell'Input e Thread Pool

L'acquisizione dei dati è delegata a un reader thread, che estrae le righe dalla pipe in ingresso e le accoda in una coda interna condivisa. I worker thread restano in attesa di elementi all'interno di tale coda, sincronizzati tramite un mutex (`mtx_t`) e variabili di condizione (`cnd_t`).

Esecuzione ed Emissione

Per ogni riga estratta, i worker thread invocano la funzione utente `mr_mapper_t`. La funzione `emit_pair`, fornita dal framework, si occupa di immettere i dati sulla pipe verso il reducer garantendo mutua esclusione mediante `mtx_lock` per evitare data race.

5. Modulo Reducer

Il modulo Reducer è incaricato dell'aggregazione dei dati emessi dai Mapper, articolandosi in una fase sequenziale di raggruppamento e una successiva riduzione concorrente.

Strutture Dati e Popolamento della Hash Table

Il thread principale del processo Reducer ascolta dalla pipe `mapper_to_reducer`, estrae i dati e aggiorna una Hash Table interna. Per gestire l'aggregazione, sono state implementate strutture come `ht`, `ht_item`, `token_chain` e `result_chain`. Questo approccio a liste concatenate risolve le collisioni di hash e permette l'inserimento in tempo costante $O(1)$. Al termine dello stream, gli item vengono ordinati lessicograficamente e accodati in una coda di lavoro.

Riduzione Concorrente

Completata l'aggregazione, il processo genera un Thread Pool di worker che consumano le chiavi univoche e le liste di valori, invocando per ciascuna la funzione `mr_reducer_t`. Il risultato viene poi trasmesso tramite `emit_result` sulla pipe finale `reducer_to_main`.

6. Main Process e Gestione Output

Il processo principale agisce come supervisore. Oltre all'istanziamento dell'infrastruttura, si occupa della scansione ricorsiva delle directory di input e del filtraggio riga per riga dei file idonei.

La memorizzazione degli output finali segue una logica indipendente dal formato: il Master traduce i byte del risultato in una stringa esadecimale. Il file risultante segue un formato posizionale testuale:

`[lunghezza_token],[token],[lunghezza_risultato_1],[hex_risultato_1],...`

7. Meccanismo di Logging

Il modulo di logging (`logs.c`) garantisce visibilità sull'esecuzione:

- **Variabili Thread-Local:** Sfrutta `thread_local` per memorizzare l'istanza globale `mr_t`, permettendo di richiamare i log da qualsiasi punto senza passare esplicitamente i puntatori.
- **Livelli e Formattazione:** Supporta i livelli INFO, ERROR, METRIC e DEBUG, riportando timestamp e PID del processo originario.
- **Persistenza:** Tutte le scritture sono incapsulate in logiche thread-safe.

8. Sviluppi Applicativi ed Esempi (Test Suite)

Per validare il framework sono stati realizzati diversi esempi:

- **Example 1 (Word Count base):** Misura il throughput e i colli di bottiglia su dataset di svariati Megabyte.
- **Example 2 (Word Count Multiplo):** Verifica la solidità dell'isolamento dei processi eseguendo istanze parallele multiple.
- **Example 3 (Edge Cases):** Verifica che puntatori nulli e directory inesistenti vengano intercettate senza generare `Segmentation Fault`.
- **Example 4 (Inverted Index):** Dimostra la flessibilità permettendo di registrare chiavi strutturate ed emettere molteplici occorrenze.

9. Conclusioni

La realizzazione di questo progetto ha permesso di consolidare i concetti fondamentali della programmazione di sistema, unendo le primitive POSIX per i processi e lo standard C11 per il multithreading. Il corretto utilizzo dei meccanismi di sincronizzazione ha garantito la stabilità del framework, replicando le dinamiche operative delle architetture di calcolo distribuito.